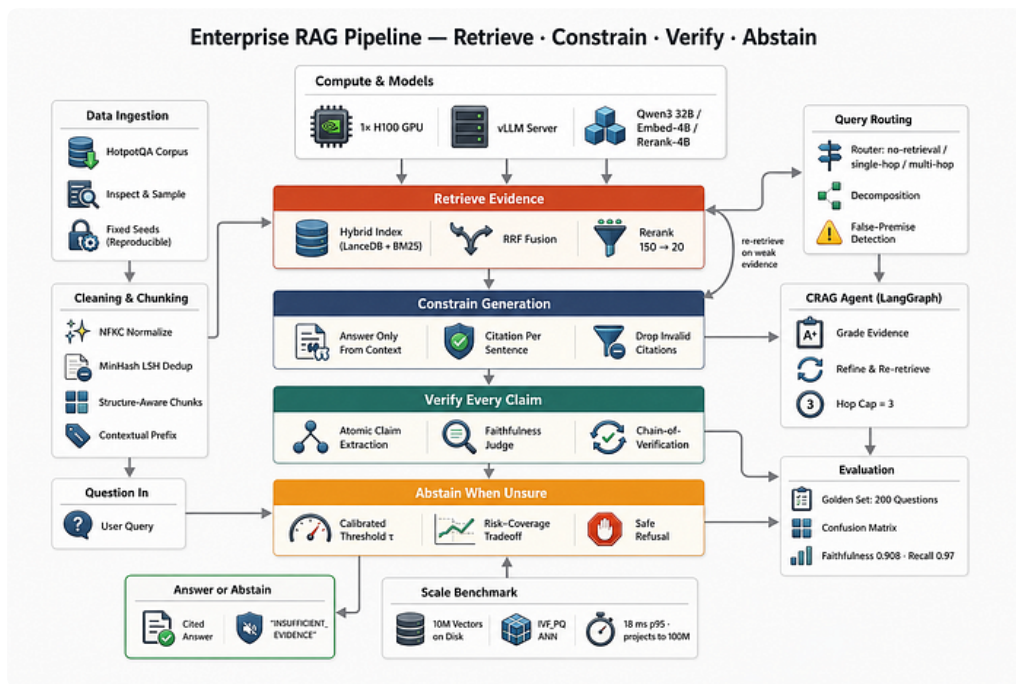




Building a RAG Pipeline for 10M+ Documents With Near-Zero Hallucination

Retrieve, constrain, verify, abstain

Fareed Khan

Published June 15, 2026

Free: No

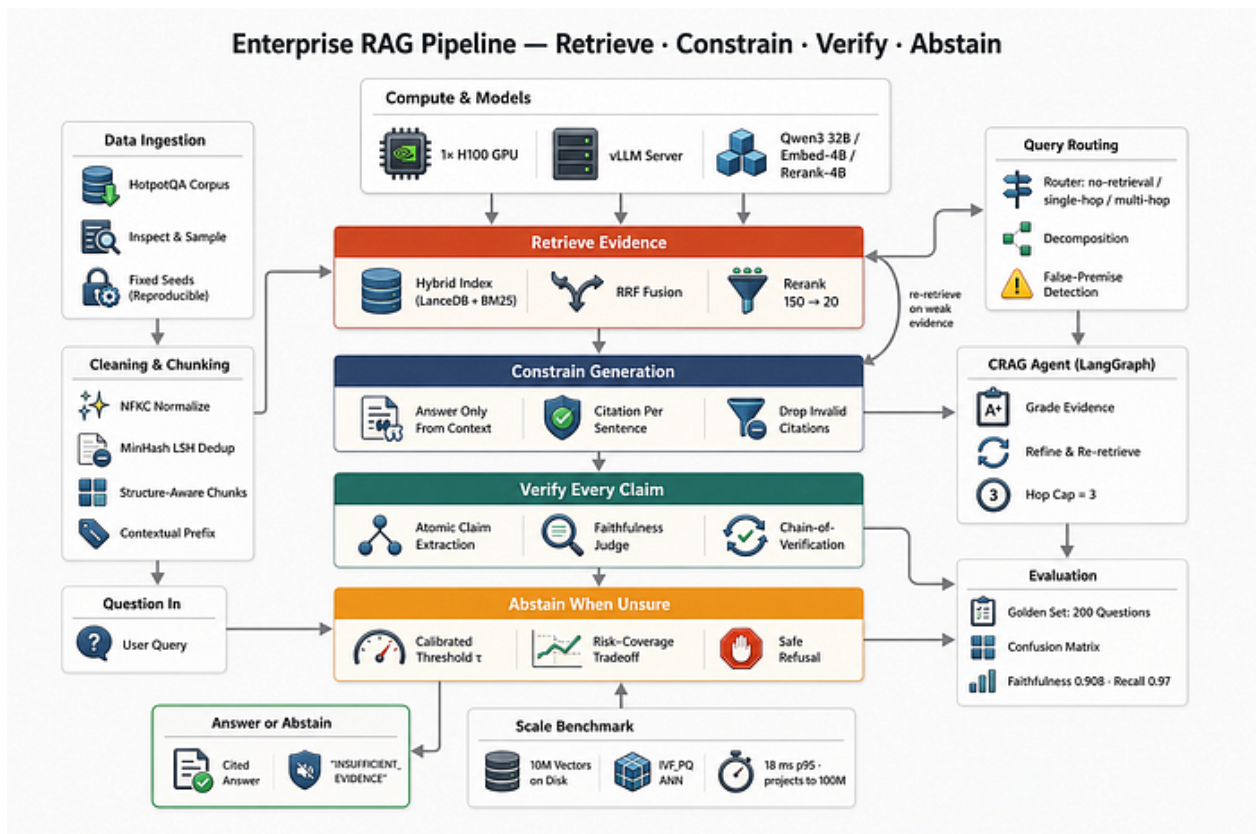
[Read on Freedium](#) · [original](#)

Contents

Table of Contents	4
Near-Zero, Not Zero	4
Setting Up the Project	5
Getting the Data	8
Cleaning the Corpus	12
Chunking and Context	13
Loading the Retrieval Models	15
Building the Hybrid Index	17
Retrieval: Fusion and Reranking	19
Reciprocal rank fusion	19
Reranking	21
Routing and Decomposition	22
Cited Generation	23
The Verification Gate	25
Knowing When to Abstain	27
The Agent	29
Does It Work?	31
The golden set	31
Hallucinations live in one cell	32
The price of safety	34
Is the judge any good?	35
Scaling to 10M+ Vectors	36
A real 10M-vector index	36
18 ms at ten million, and a 100M projection	38
Where the time goes	39
Scope and What Comes Next	40

Read this story for free: [link](https://freedium-mirror.cfd/https://medium.com/@fareedkhandev/788e4b5b7f25?source=friends_link&sk=0030c7cf6410deac59340019bc892d11) (https://freedium-mirror.cfd/https://medium.com/@fareedkhandev/788e4b5b7f25?source=friends_link&sk=0030c7cf6410deac59340019bc892d11)

The more documents you put into a RAG system, the more ways it has to make things up, and as the corpus grows into the millions, toward 10M and beyond, that hallucination problem only gets worse. To keep answers trustworthy at that scale, you need a pipeline where the agent checks its own evidence and cites every claim it makes, the same idea behind the citations that Claude uses.



Here is everything the pipeline contains, and we build it top to bottom, one component at a time:

- **Set up and get the data:** download the corpus, inspect its size and a real sample, and fix every seed so the run is reproducible.
- **Clean and chunk:** normalize the text, drop near-duplicates with MinHash LSH, and cut it into structure-aware chunks with a one-line context prefix.
- **Build a hybrid index:** store every chunk as a dense vector in LanceDB and a sparse BM25 posting, on disk so it scales to 10M+ vectors.
- **Retrieve and rerank:** fuse the dense and sparse rankings with reciprocal rank fusion, then rerank 150 candidates down to 20.
- **Route and decompose:** classify each question and split multi-hop ones into sub-questions before retrieving.
- **Generate with citations:** answer strictly from the context with a citation on every sentence, or emit an abstain token.

- **Verify every claim:** split the answer into atomic claims and check each one against its cited text with a faithfulness judge.
- **Abstain when unsure:** fold the signals into one calibrated decision and refuse when the support is not there.
- **Wire the agent:** connect it all into a self-correcting CRAG loop that re-retrieves on weak evidence.
- **Evaluate and scale:** score hallucination on a 200-question golden set, then benchmark the index to a real 10M vectors and project to 100M.

All the code is available in my GitHub repository (Theory + Code):

GitHub - FareedKhan-dev/rag-zero-hallucinations: Handling 10M+ docs using RAG with zero... (<https://github.com/FareedKhan-dev/rag-zero-hallucinations>)

Handling 10M+ docs using RAG with zero hallucinations - GitHub - FareedKhan-dev/rag-zero-hallucinations: Handling 10M+...

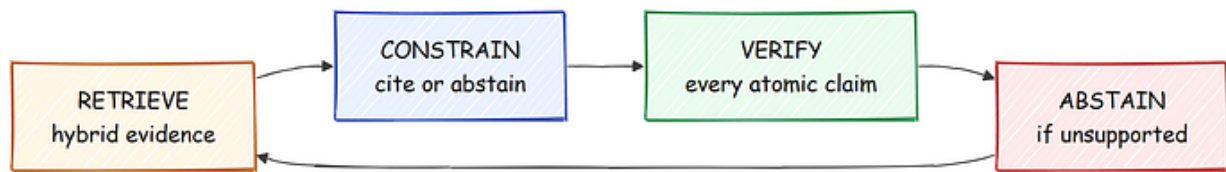
Table of Contents

- [Near-Zero, Not Zero](#)
- [Setting Up the Project](#)
- [Getting the Data](#)
- [Cleaning the Corpus](#)
- [Chunking and Context](#)
- [Loading the Retrieval Models](#)
- [Building the Hybrid Index](#)
- [Retrieval: Fusion and Reranking](#) • [Reciprocal rank fusion](#) • [Reranking](#)
- [Routing and Decomposition](#)
- [Cited Generation](#)
- [The Verification Gate](#)
- [Knowing When to Abstain](#)
- [The Agent](#)
- [Does It Work?](#) • [The golden set](#) • [Hallucinations live in one cell](#) • [The price of safety](#) • [Is the judge any good?](#)
- [Scaling to 10M+ Vectors](#) • [A real 10M-vector index](#) • [18 ms at ten million, and a 100M projection](#) • [Where the time goes](#)
- [Scope and What Comes Next](#)

Near-Zero, Not Zero

The problem we have to solve is not "make the model smarter." A bigger model still guesses when retrieval comes back empty, because guessing is what generation does.

So instead of chasing a perfect model, we wrap an ordinary one in a system that has only one safe failure mode. When the evidence is missing, the right output is not a fluent guess, it is an abstention.



That gives us four control layers, and every section below is one of them.

1. **Retrieve the right evidence:** hybrid dense plus BM25 search, contextual chunks, and reranking.
2. **Constrain generation:** answer only from the context, cite passage ids for every sentence, or abstain.
3. **Verify every atomic claim:** check each claim against the cited text with a faithfulness judge.
4. **Abstain:** when claim support or retrieval confidence falls below a calibrated threshold.

We are after two goals at the same time. The first is **trust**, which means near-zero hallucination on the questions we choose to answer.

The second is **scale**, which means the retrieval backbone has to hold 10M+ vectors and still answer in milliseconds. The first goal needs the verification logic, the second needs the index.

We build both.

Setting Up the Project

Before any of the logic, we set up the project. The plan is to import the libraries, fix every random seed so the run is reproducible, check the one GPU we are given, point a thin client at the generator, and freeze the config so a headless run behaves the same every time.



First the imports and a single function that seeds every random number generator we will touch.

```

import json, os, random, subprocess, time
from dataclasses import dataclass, asdict, field
import numpy as np

def set_determinism(seed: int) -> None:
    """Seed every RNG we touch so runs are reproducible."""
    random.seed(seed)
    np.random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    try:
        import torch
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    except Exception:
        pass

set_determinism(42)

```

I fix the seed up front because a RAG evaluation that is not reproducible is not an evaluation, and because the whole point of this blog is to trust the numbers at the end. The notebook is also parameterized, so one cell resolves the run profile and prints what we are about to build on.

```

#### OUTPUT ####
profile=FULL slice=20000 eval=100+100 artifacts=/mnt/data/artifacts

```

This is the full run, twenty thousand passages and a hundred plus a hundred evaluation questions, not the tiny smoke profile I use first to shake out code errors cheaply. We are on a single GPU, so VRAM is a hard budget, not a runtime surprise. We read the card with `nvidia-smi` and assert we are where we expect to be.

```
def gpu_report() -> dict:
    """Return GPU name / memory / driver and assert we are on an 80GB H100."""
    name = _smi("name")[0]
    total = float(_smi("memory.total")[0]) / 1024.0          # GiB
    rep = {"name": name, "total_gb": round(total, 1),
          "free_gb": round(float(_smi("memory.free")[0]) / 1024.0, 1),
          "driver": _smi("driver_version")[0]}
    print(json.dumps(rep, indent=2))
    assert "H100" in name and total >= 79          # one 80GB H100, nothing smaller
    return rep
#### OUTPUT ####
{
  "name": "NVIDIA H100 PCIe",
  "total_gb": 79.6,
  "free_gb": 32.8,
  "driver": "570.195.03"
}
```

We are on one **NVIDIA H100 with 80 GB**, and the host around it has 180 GB of RAM and a 750 GB NVMe disk, which matters later when the index grows. The 32B generator does not live in this notebook.

It lives in a separate vLLM server, and we talk to it with a small OpenAI-compatible client. Keeping it warm in its own process means we can re-run this notebook many times without ever reloading it.

```
class LocalLLM:
    """Thin client for the warm vLLM OpenAI-compatible server."""

    def __init__(self, endpoint: str, model: str, thinking: bool = False):
        self.endpoint, self.model, self.thinking = endpoint.rstrip("/"), model, thinking

    def chat(self, system: str, user: str, temperature: float = 0.0, max_tokens: int = 512) -> str:
        body = {"model": self.model, "temperature": temperature, "max_tokens": max_tokens,
              "messages": [{"role": "system", "content": system},
                          {"role": "user", "content": user}]}
        if not self.thinking:          # Qwen3: skip the <think> trace for low
latency
            body["chat_template_kwargs"] = {"enable_thinking": False}
        r = requests.post(f"{self.endpoint}/chat/completions", json=body, timeout=120)
        r.raise_for_status()
        return r.json()["choices"][0]["message"]["content"]

llm = LocalLLM("http://localhost:8000/v1", "Qwen/Qwen3-32B")
print(f"[llm] up={llm.is_up()}")
#### OUTPUT ####
[llm] up=True
```


The server is up. The last setup step is to freeze every knob into one config object and print it, so the numbers driving the rest of the blog are all in one place.

```
#### OUTPUT ####
{
  "gen_model": "Qwen/Qwen3-32B",
  "embed_offline": "Qwen/Qwen3-Embedding-4B",
  "rerank_model": "Qwen/Qwen3-Reranker-4B",
  "chunk_tokens": 256, "chunk_overlap": 32,
  "retrieve_k": 150, "rerank_top_n": 20, "rrf_k": 60,
  "max_hops": 3, "crag_ok": 0.7, "crag_bad": 0.4,
  "tau_claim": 0.3, "tau_abstain": 0.3, "seed": 42
}
```

We retrieve 150 candidates, rerank down to 20, allow the agent up to 3 corrective hops, and set two support thresholds at 0.3 that we will calibrate later. The generator is **Qwen3-32B**, the embedder and reranker are the 4B Qwen3 models, and the faithfulness judge is the 32B itself.

I keep the generator at temperature 0 with the thinking trace off, because I want reproducible, low-latency answers, and because sampling is one more place a model can drift away from the evidence I gave it. Every model here is a local open-weight Qwen3, which I have to do because the entire premise is that no document and no query leaves this machine, which is exactly what makes a pipeline like this usable on a private corpus.

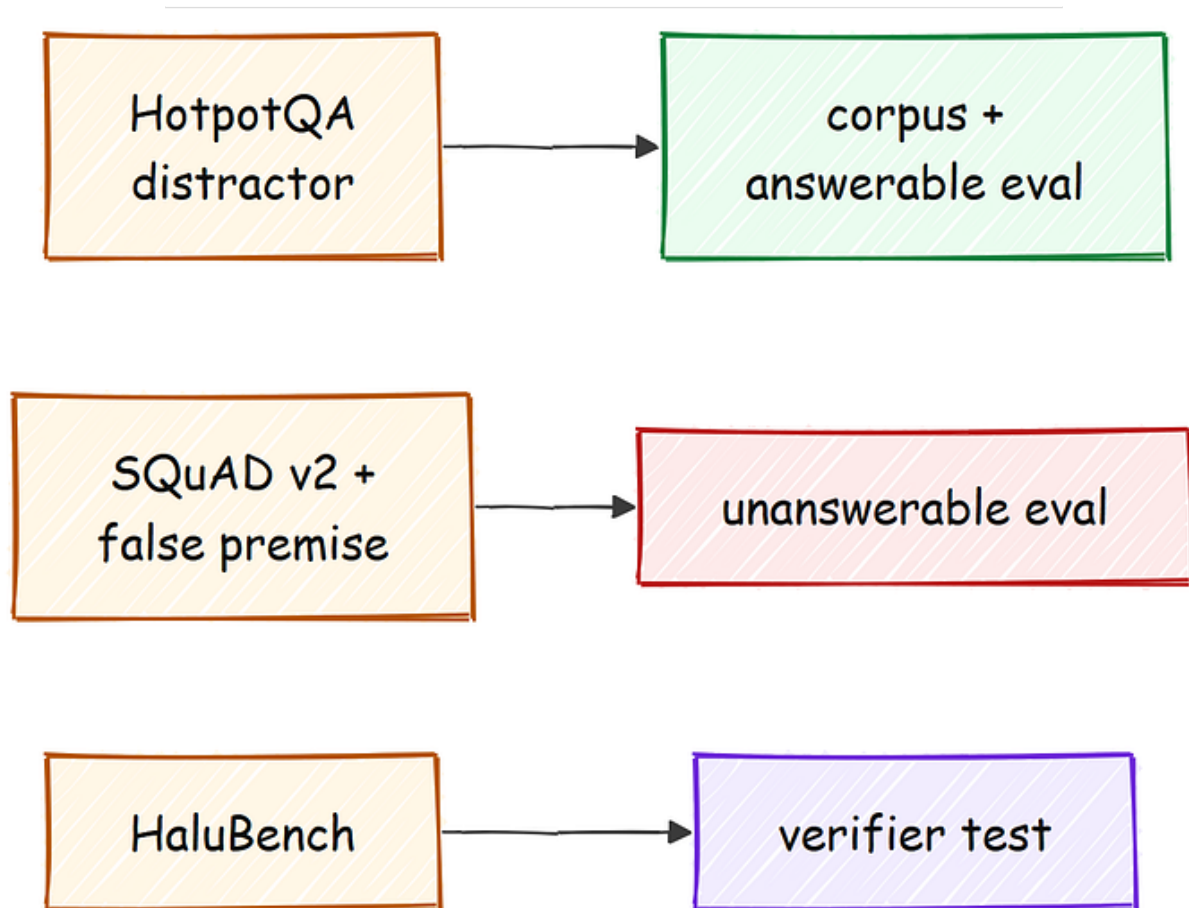
With the tools ready, we can go get some data.

Getting the Data

A pipeline is only as good as the corpus under it, so the first real step is to download a dataset and look at it. I went with **HotpotQA** in its distractor setting for two reasons.

Every question ships with sentence-level gold supporting facts, which is the cleanest way to score retrieval recall later, and its bundled Wikipedia paragraphs give me a real corpus for free. For the other side of the test I pull **SQuAD v2** impossible questions and hand-write a handful of false-premise questions, because the only way to measure hallucination is to ask things the corpus cannot answer and check that the system stays quiet.

A third set, **HaluBench**, comes in near the end purely to validate the verifier itself. HotpotQA is the corpus we build and search.



```
from datasets import load_dataset

def load_hotpotqa(split: str = "validation"):
    # datasets 3.x wants the namespaced repo id
    return load_dataset("hotpotqa/hotpot_qa", "distractor", split=split, cache_dir=DS_CACHE)

hotpot = load_hotpotqa()
print(f"[data] hotpotqa(validation) = {len(hotpot)} questions")
#### OUTPUT ####
[data] hotpotqa(validation) = 7405 questions
```

That is **7,405 questions**, and bundled with each one are the Wikipedia paragraphs it was drawn from. We define what a passage and a question look like, then a builder that unions every question's context paragraphs into a corpus while keeping track of which passages are gold evidence.

```

@dataclass
class Passage:
    id: str
    title: str
    text: str
    is_gold_for: list[str] = field(default_factory=list)  # question ids this is gold for

@dataclass
class QAItem:
    qid: str
    question: str
    answer: str
    answerable: bool
    gold_titles: list[str] = field(default_factory=list)
    gold_sentences: list[str] = field(default_factory=list)
    qtype: str = ""  # bridge | comparison | unanswerable | false_premise

class CorpusBuilder:
    """Build a passage corpus + QA items from HotpotQA distractor contexts."""

    def build(self, qa, n_passages: int):
        passages, qa_items = {}, []
        for ex in qa:
            gold = list(dict.fromkeys(ex["supporting_facts"]["title"]))  # gold evidence
            titles
            for t, ss in zip(ex["context"]["title"], ex["context"]["sentences"]):
                para = " ".join(s.strip() for s in ss).strip()
                if len(para) < 40:
                    continue
                p = passages.setdefault(_pid(t, 0), Passage(_pid(t, 0), t, para))
                if t in gold:
                    p.is_gold_for.append(ex["id"])
            # (the full builder also records each question's gold supporting sentences)
            qa_items.append(QAItem(ex["id"], ex["question"], ex["answer"], True,
                                   gold_titles=gold, qtype=ex.get("type", "")))
            if len(passages) >= n_passages:
                break
        return list(passages.values()), qa_items

corpus, qa_items = CorpusBuilder().build(hotpot, SLICE_SIZE)
print(f"[corpus] passages={len(corpus)}  qa_items={len(qa_items)}  "
      f"gold-bearing passages={sum(1 for p in corpus if p.is_gold_for)}")
#### OUTPUT ####
[corpus] passages=20007  qa_items=2073  gold-bearing passages=4072

```

We now hold **20,007 passages** and 2,073 questions, with 4,072 passages marked as gold evidence for some question. Before building anything on top of it, we should actually look at the data, both the size distribution and one real example.

```
import pandas as pd

tok_lens = [len(p.text.split()) for p in corpus]
print(pd.Series(tok_lens, name="passage_word_count").describe().round(1).to_string())

ex = qa_items[0]
print(f"\nSample question:\n Q: {ex.question}\n A: {ex.answer}    (type={ex.qtype})")
print(f"  gold titles: {ex.gold_titles}")
for s in ex.gold_sentences:
    print(f"    - {s}")
#### OUTPUT ####
count      20007.0
mean         89.2
std          53.4
min           7.0
25%          54.0
50%          80.0
75%         113.0
max         1378.0

Sample question:
Q: Were Scott Derrickson and Ed Wood of the same nationality?
A: yes    (type=comparison)
gold titles: ['Scott Derrickson', 'Ed Wood']
- Scott Derrickson (born July 16, 1966) is an American director, screenwriter and producer.
- Edward Davis Wood Jr. was an American filmmaker, actor, writer, producer, and director.
```

Passages run about **89 words on average**, short enough that a couple fit in a prompt and long enough to carry a fact. The sample is a comparison question, "Were Scott Derrickson and Ed Wood of the same nationality?", and its two gold sentences already contain the answer, that both men were American.

This is the question we will follow through every stage of the blog, because watching one real question travel the whole pipeline makes each component concrete. The two strata are already visible here.

Answerable questions like this one let me measure whether the right evidence comes back, and the unanswerable questions I add later are how I measure hallucination, because a system that answers a question with no support in the corpus is a system that makes things up.

Cleaning the Corpus

Garbage in means hallucinations out, so before we index anything we clean the text. Two cheap steps pay off out of proportion. Normalization makes the tokenizer behave the same on every passage, and near-duplicate removal stops copied or forwarded passages from crowding the top results and inflating retrieval without adding any new evidence.



```
import re, unicodedata

def normalize_text(s: str) -> str:
    s = unicodedata.normalize("NFKC", s)      # canonical unicode form
    s = s.replace(" ", "")                  # drop soft hyphens
    s = re.sub(r"[ \t]+", " ", s)           # collapse runs of spaces
    return s.strip()
```

I run NFKC normalization first because BM25 tokenizes on raw characters, so a ligature or a run of stray spaces would split one word into two or merge two into one, and quietly hurt recall. On a messy string the function does exactly what we want.

```
#### OUTPUT ####
>>> normalize_text("the final report\twas ready")
'the final report was ready'
```

The ligature "fi" becomes a plain "fi" and the tab and runs of spaces collapse to single spaces, so two passages that differ only in invisible characters now tokenize identically.

The deduper is the interesting part. I have to choose an approximate method like MinHash LSH rather than comparing every pair, because exact pairwise comparison is quadratic and would never finish at corpus scale, while MinHash with an LSH index finds near-duplicates in roughly linear time.

Dropping them serves both goals at once. It keeps the index smaller as we head toward 10M vectors, and it stops three copies of one paragraph from crowding the top results, which is a quiet way a retriever feeds the model redundant context and tempts it to over-trust a single source.

```

class Deduper:
    """Drop near-duplicate passages via MinHash LSH over word shingles."""

    def __init__(self, threshold: float = 0.9, num_perm: int = 64):
        self.threshold, self.num_perm = threshold, num_perm

    def fit_transform(self, passages: list[Passage]):
        lsh = MinHashLSH(threshold=self.threshold, num_perm=self.num_perm)
        kept, dropped = [], 0
        for p in passages:
            m = self._mh(p.text)
            if lsh.query(m):          # a near-duplicate is already kept
                dropped += 1
                continue
            lsh.insert(p.id, m)
            kept.append(p)
        return kept, {"kept": len(kept), "dropped_near_dup": dropped}

#### OUTPUT ####
{
  "kept": 19987,
  "dropped_near_dup": 19,
  "input": 20007,
  "after_quality": 20006,
  "after_dedup": 19987
}

```

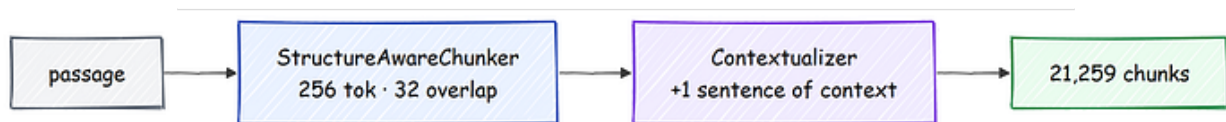
We keep **19,987 passages** after dropping 19 near-duplicates and one short fragment. This corpus is a curated slice, but the cleaning step is exactly what you run unchanged whether the input is twenty thousand passages or twenty million.

Chunking and Context

Now we cut passages into chunks. Fixed-size chunking is the easy choice and the wrong one, because it cuts an entity-bearing sentence away from the context that disambiguates it, which is fatal for multi-hop questions.

So we pack whole sentences up to a token budget with a small overlap, and we count tokens with the generator's own tokenizer so the budget matches what the model will actually see. This is a hallucination problem hiding inside a chunking detail.

If a chunk overflows the budget and gets silently truncated, the one sentence that held the answer can vanish, and the question then looks unanswerable for no real reason, so I would rather respect sentence boundaries and pay for a few extra chunks.



```

class StructureAwareChunker:
    def __init__(self, tokenizer, target_tokens: int = 256, overlap: int = 32):
        self.tok, self.target, self.overlap = tokenizer, target_tokens, overlap

    def chunk(self, passage: Passage) -> list[Chunk]:
        sents = split_sentences(passage.text) or [passage.text]
        chunks, cur, cur_tok = [], [], 0
        for s in sents:
            st = self._ntok(s)
            # start a new chunk once adding this sentence would blow the token budget
            if cur and cur_tok + st > self.target:
                chunks.append(self._make(passage, cur))
                # carry the trailing sentence forward so chunks overlap
                cur, cur_tok = ([cur[-1]], self._ntok(cur[-1])) if self.overlap else ([], 0)

            cur.append(s)
            cur_tok += st
        if cur:
            chunks.append(self._make(passage, cur))
        return chunks

#### OUTPUT ####
[chunk] 19987 passages -> 21259 chunks (tokens: mean=125 p95=236)
  
```

That gives us **21,259 chunks** at a mean of 125 tokens, comfortably under the 256 budget. There is one more problem to solve before indexing.

A chunk like "revenue grew 3 percent that quarter" is unsearchable on its own, because whose revenue and which quarter are gone. So we prepend a one-line situating sentence to each chunk before indexing, which is the contextual retrieval idea, except we write that sentence with our local Qwen3 instead of a hosted model.

```

CONTEXTUALIZE_PROMPT = (
    "Here is a document titled '{title}':\n<document>\n{doc}\n</document>\n\n"
    "Here is a chunk from it:\n<chunk>\n{chunk}\n</chunk>\n\n"
    "Give a short, single-sentence context (<=25 words) that situates this chunk "
    "within the document so it can be retrieved on its own. Answer with the sentence "
    "only."
)
  
```

The method fans the per-chunk calls out across a thread pool, because the calls are independent and vLLM batches them server-side, which makes this far faster than going one chunk at a time. We also checkpoint the result so a rerun skips this whole step.

```
class Contextualizer:
    def contextualize(self, chunks, doc_lookup, workers: int = 32):
        def _one(c):
            user = CONTEXTUALIZE_PROMPT.format(title=c.title,
                                                doc=doc_lookup.get(c.passage_id, c.text)[:4000],
                                                chunk=c.text)

            ctx = self.llm.chat("You write concise retrieval context.", user, max_tokens=64).strip()
            c.contextual_text = (ctx + "\n" + c.text) if ctx else c.text  # prefix, keep original

        with ThreadPoolExecutor(max_workers=workers) as ex:
            list(ex.map(_one, chunks))  # 32 in flight at once
        return chunks

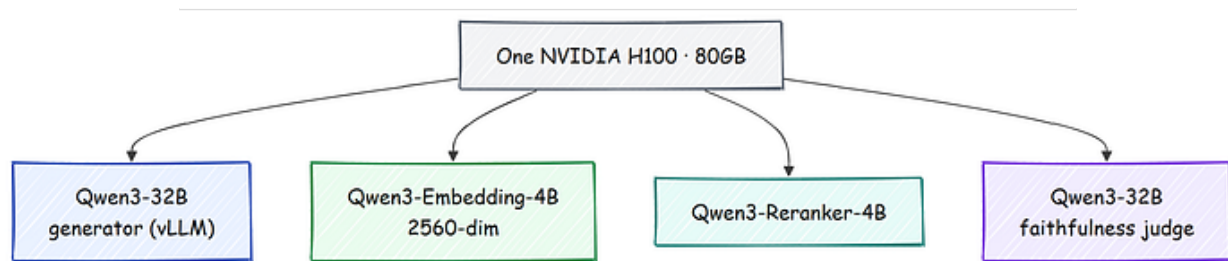
#### OUTPUT ####
Before:
Ed Wood is a 1994 American biographical period comedy-drama film directed and produced by Tim Burton, and starring Johnny Depp as cult filmmaker Ed Wood...

After (context-prefixed):
This chunk introduces the 1994 film *Ed Wood*, directed by Tim Burton, and outlines its main subject and cast.
Ed Wood is a 1994 American biographical period comedy-drama film...
```

The extra sentence is cheap, one short generation per chunk, and it tells the retriever what this chunk is about even when the chunk text alone would be ambiguous. That lift is most of why recall ends up so high. Recall is the foundation of the whole hallucination story, because the verifier downstream can only ground an answer in evidence that retrieval actually found, so every point of recall I buy here is a question I get to answer instead of refuse.

Loading the Retrieval Models

With the chunks ready, we load the models that turn them into searchable evidence and later check the answers. Three models share this GPU alongside the generator, so we snapshot VRAM after each load and stay under budget. We load the **reranker** and the **faithfulness judge** here, and the embedder a little later, only when we build the index.



We do not want to discover an overrun as an out-of-memory crash three steps later, so each load logs the whole-GPU number from `nvidia-smi` and the kernel-only number from `torch`.

```
def vram_snapshot(tag: str) -> dict:
    """Log GPU-wide and kernel-only VRAM after each load step."""
    kernel = round(torch.cuda.memory_allocated() / 1024**3, 2) # this kernel only
    used = round(float(_smi("memory.used")[0]) / 1024.0, 2) # whole GPU, both
    processes
    print(f"[vram] {tag:22} gpu_used={used}GB kernel={kernel}GB")
    return {"tag": tag, "gpu_used_gb": used, "kernel_gb": kernel}
```

The reranker is a small causal model used as a yes-or-no judge. Each query and document pair is wrapped in a fixed template, and the score is read straight from the next-token logits, so reranking is one forward pass per candidate. I load a dedicated cross-encoder reranker instead of trusting the embedding scores because the embedder compresses a whole passage into one vector, which is fast enough to scan the corpus but blurs the difference between a passage that merely mentions the entities and one that actually answers the question, and that difference is precisely what keeps the wrong evidence out of the prompt and out of the answer.

```
class Qwen3Reranker:
    """Scores a (query, doc) pair by the probability the model puts on the 'yes' token."""

    @torch.no_grad()
    def score(self, query: str, docs: list[str], batch_size: int = 16) -> list[float]:
        out = []
        for i in range(0, len(docs), batch_size):
            batch = [self._fmt(query, d) for d in docs[i:i + batch_size]]
            enc = self.tok(batch, return_tensors="pt", padding=True,
                           truncation=True, max_length=1024).to(self.model.device)
            logits = self.model(**enc).logits[:, -1, :] # last-token logits
            yn = logits[:, [self.no_id, self.yes_id]] # compare 'no' against 'yes'
            probs = torch.softmax(yn.float(), dim=-1)[:, 1] # keep P('yes')
            out.extend(probs.cpu().tolist())
        return out
```

The faithfulness judge is the 32B generator itself, prompted to return a single support score for a claim against some context. I made the judge the local 32B because faithfulness checking in RAG means reading one claim against several long passages at once, which is exactly where a small sentence-pair NLI model gets brittle, and because this judge is the single component that turns a confident wrong answer into an abstention.

It is the heart of the near-zero hallucination claim, so I would rather spend the strongest model I have on it. An NLI cross-encoder and MiniCheck are still wired in as lighter alternatives, but this run uses the LLM judge.

```
JUDGE_PROMPT = (
    "You are a strict fact-checker. Decide whether the CONTEXT supports the CLAIM.\n\n"
    "CONTEXT:\n{context}\n\nCLAIM: {claim}\n\n"
    "Output ONLY a number: 1.0 if the context clearly states or entails the claim, "
    "0.0 if it contradicts or does not mention it, or a value in between."
)

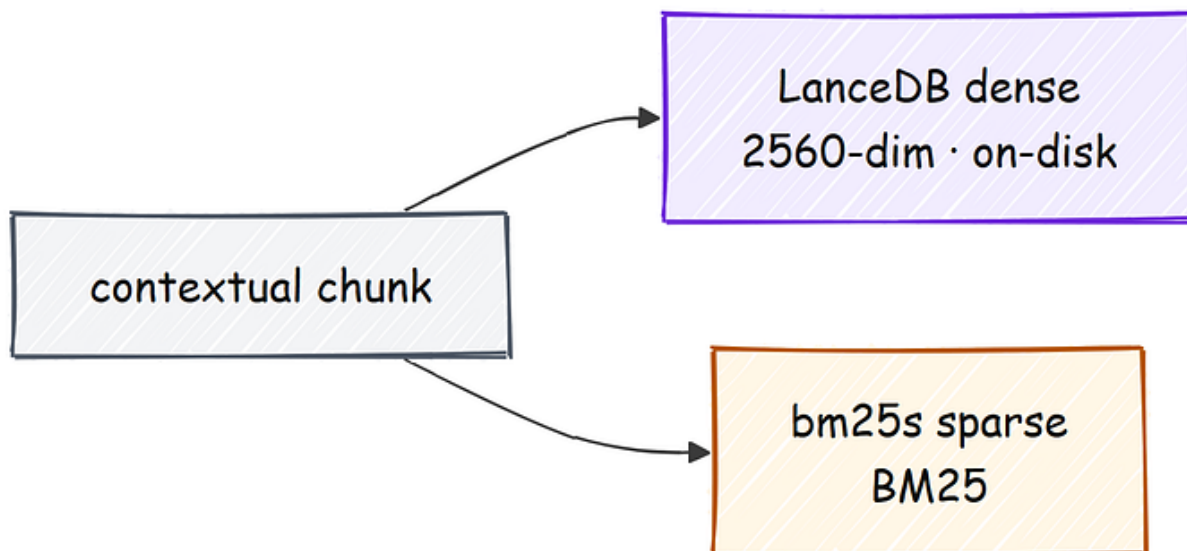
class JudgeVerifier:
    def _score(self, claim: str, context: str) -> float:
        out = self.llm.chat("You are a strict faithfulness grader.",
                            JUDGE_PROMPT.format(context=context[:6000], claim=claim), max_
tokens=8)
        m = re.search(r"[01](?:\.\d+)?", out)
        return min(1.0, float(m.group())) if m else 0.0
#### OUTPUT ####
[vram] reranker          gpu_used=54.3GB  kernel=7.49GB
[verifier] using the local LLM as faithfulness judge
[vram] whole-GPU used=54.3GB / 80.0GB (need >= 3.0GB headroom)
```

The whole stack sits at **54.3 GB of the 80 GB** the H100 gives us, which leaves headroom for the index work that comes next. The judge needs no extra VRAM, because it reuses the generator already running in the vLLM server. Everything stayed on one box, and nothing reached out to an external API.

Building the Hybrid Index

Now we index, and the problem here is that no single retriever is enough. Dense embeddings catch paraphrase, which is what you want when the question and the answer use different words.

BM25 catches exact tokens like names, ids, and numbers, which is exactly what dense models blur. So we index both, keyed by chunk id, over the contextualized text.



We load the embedder just for indexing, embed every chunk, and free it before serving queries with a smaller online embedder. The vectors are normalized so cosine similarity is a plain dot product.

```
def embed_texts(embedder, texts, is_query: bool = False) -> np.ndarray:
    kw = {"normalize_embeddings": True, "convert_to_numpy": True, "batch_size": 64}
    if is_query:
        # Qwen3-Embedding wants a query instruction prompt
        kw["prompt_name"] = "query"
    return embedder.encode(texts, **kw).astype("float32")
```

Loading the query embedder is the last thing to push VRAM, and the snapshot shows where we land.

```
#### OUTPUT ####
[vram] embedder(online)      gpu_used=61.85GB  kernel=15.04GB
```

That peak is about 62 GB of the 80, still inside budget, and I free the heavier offline embedder right after indexing so only the small online one stays resident for queries. I have to choose LanceDB for the dense side because it is embedded and on-disk on NVMe with no server to run, which means the same code path holds an index far larger than RAM, and that one property is what lets this design reach 10M+ vectors later without changing a line.

The dense side is a thin wrapper over it. The only subtlety is turning cosine distance back into a similarity in the zero-to-one range.

```
class LanceVectorStore:
    def search(self, qvec: np.ndarray, k: int) -> list[tuple[str, float]]:
        res = self.tbl.search(qvec).metric("cosine").limit(k).to_list()
        # cosine _distance is in [0, 2], so convert it to a similarity in [0, 1]
        return [(r["id"], 1.0 - r["_distance"] / 2.0) for r in res]
```

I keep a lexical bm25s index alongside the vectors because dense embeddings are exactly the thing that blurs a rare name, an id, or a number into its neighbors, and those are often the tokens a factual question turns on, so the sparse side is my insurance against a confident answer built on a near-miss passage. The sparse side stems the query the same way it stemmed the documents, then returns the top matches by BM25 score.

```
class BM25Index:
    def search(self, query: str, k: int) -> list[tuple[str, float]]:
        q = bm25s.tokenize(query, stemmer=self.stemmer)
        idx, scores = self.retriever.retrieve(q, k=min(k, len(self.ids)))
        return [(self.ids[int(i)], float(s)) for i, s in zip(idx[0], scores[0])]

#### OUTPUT ####
[index] LanceDB on-disk: /mnt/data/artifacts/lancedb | bm25 over 21259 chunks
```

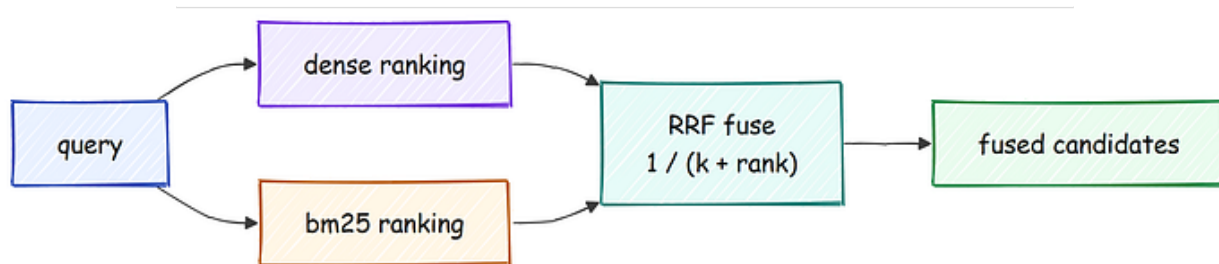
The whole index for these 21,259 chunks is about **11.1 MB** on disk, which is tiny, but the point is the shape, not the size. LanceDB keeps the vectors on NVMe rather than in RAM, so the same code path holds an index that is far larger than memory. That is the property we lean on at the end of the blog when we push this design to ten million vectors.

Retrieval: Fusion and Reranking

Reciprocal rank fusion

We have two ranked lists now, one dense and one sparse, and we have to combine them. The trap is that their scores are not comparable, because a BM25 score and a cosine similarity live on different scales.

Reciprocal rank fusion sidesteps that completely. It ignores the scores and uses only the rank, giving each result a weight of one over k plus its rank, then sums those weights across both lists.



```

def rrf_fuse(rankings: list[list[str]], k: int = 60) -> list[tuple[str, float]]:
    scores: dict[str, float] = {}
    for ranking in rankings:
        for rank, cid in enumerate(ranking):
            # a later rank adds less, and no score normalization is needed
            scores[cid] = scores.get(cid, 0.0) + 1.0 / (k + rank + 1)
    return sorted(scores.items(), key=lambda x: -x[1])
  
```

It is easier to see than to describe. Take two short rankings where the dense list and the sparse list disagree, and watch what fusion does.

```

#### OUTPUT ####
>>> rrf_fuse([["a", "b", "c"], ["b", "c", "a"]])
[('b', 0.03252), ('a', 0.03227), ('c', 0.03200)]
  
```

Document **b** wins even though neither list put it first, because it sits near the top of both. That is the whole point.

A result two different retrievers agree on rises above a result only one of them loved. The reason we fuse at all is that the two retrievers fail in different ways.

Dense search misses a rare proper noun that does not sit near anything it has seen in embedding space, and sparse search misses a paraphrase that shares no words with the query, so fusing them recovers the documents each one alone would have dropped. The retriever ties them together, embedding the query once, running both searches at the same width, and fusing the two id rankings into one.

```

class HybridRetriever:
    def retrieve(self, query: str, k: int) -> list[RetrievedChunk]:
        qvec = embed_texts(self.embedder, [query], is_query=True)[0]
        dense = self.vec.search(qvec, k) # dense catches paraphrase and meaning
        sparse = self.bm25.search(query, k) # sparse catches exact names, ids, numbers
        fused = rrf_fuse([i for i, _ in dense], [i for i, _ in sparse], self.rrf_k)[:k]
        return [c for c in (self._mk(cid, s, "hybrid") for cid, s in fused) if c]
  
```

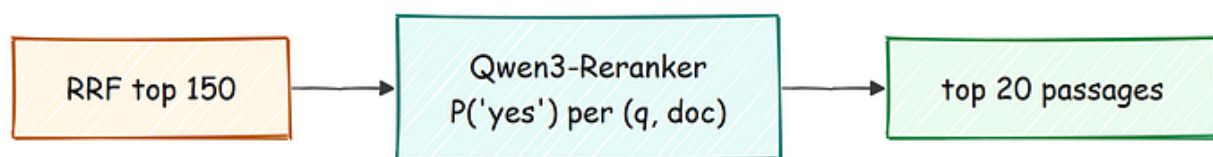
The fused list is our recall stage, deliberately wide at 150 candidates, because the next stage is where we trade that recall for precision.

Reranking

Recall is cheap and precision is expensive, so we run them in that order. The reranker we loaded earlier reads the query and a candidate together and scores how well they match, which is far more accurate than the bi-encoder embeddings but far too slow to run over the whole corpus.

Running it over only the 150 fused candidates is the sweet spot. Running the expensive model on 150 candidates instead of the whole corpus is also a scaling decision, because that cost is fixed at 150 pairs whether the index holds twenty thousand chunks or ten million.

A thin stage wraps the model, scores every candidate, and keeps the top twenty.



```

class RerankerStage:
    def rerank(self, query, cand, top_n):
        scores = self.reranker.score(query, [c.text for c in cand])
        ranked = sorted(zip(cand, scores), key=lambda x: -x[1])[:top_n]
        out = []
        for c, s in ranked:
            c.score, c.source = float(s), "reranked"
            out.append(c)
        return out
  
```

We can prove the lift by measuring passage recall against the HotpotQA gold titles, dense alone, then hybrid, then reranked, on our running example.

```

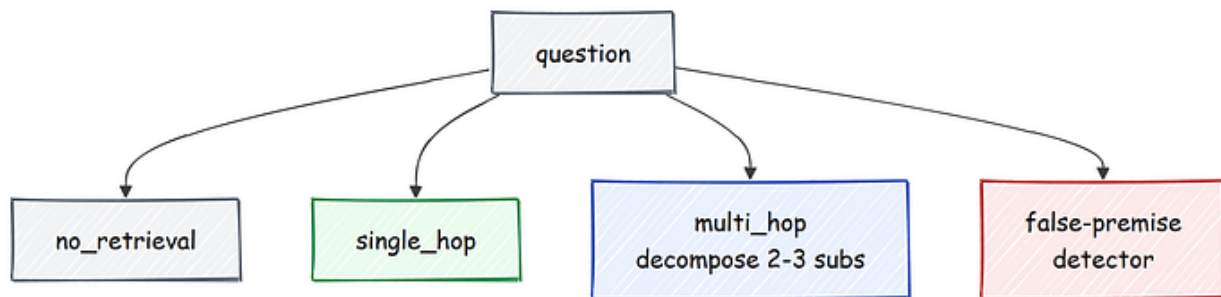
#### OUTPUT ###
Q: Were Scott Derrickson and Ed Wood of the same nationality?
gold titles: ['Scott Derrickson', 'Ed Wood']
recall@20: dense=1.00 hybrid=1.00 reranked=1.00
top-3 reranked:
[a9ec406223bd] (0.999) Scott Derrickson
[2d2201c92ac5] (0.996) Ed Wood
[b7dbb0e190b4] (0.796) Ed Wood (film)
  
```

Both gold passages land in the top three with reranker scores of 0.999 and 0.996, while the less relevant film article sits lower at 0.796. Across the full evaluation this retrieval stack reaches **0.97 context recall**, which means the evidence is almost always there when the question is answerable.

Retrieval is solved. Everything after this is about not abusing it.

Routing and Decomposition

Not every query deserves the full pipeline. A greeting needs no retrieval, a simple lookup needs one hop, and a comparison needs several. So the first thing the agent does is route the question into one of three labels, which lets us spend compute only where it helps.



```

ROUTER_PROMPT = (
    "Classify the question into exactly one label:\n"
    "- no_retrieval: greetings/opinions or questions no document corpus could answer\n"
    "- single_hop: answerable by finding one fact\n"
    "- multi_hop: needs combining facts from multiple documents\n"
    "Question: {q}\nReply with only the label."
)

class QueryRouter:
    LABELS = {"no_retrieval", "single_hop", "multi_hop"}

    def route(self, query: str) -> str:
        out = self.llm.chat("You are a precise query classifier.",
                             ROUTER_PROMPT.format(q=query), max_tokens=8).strip().lower()
        for lbl in self.LABELS:
            if lbl in out:
                return lbl
        return "single_hop"  # a safe default if the model is chatty
  
```

The decomposer and the false-premise check are just as small. The decomposer asks for two or three self-contained sub-questions, and the detector asks a blunt yes-or-no question about whether the query assumes something that may not be true.


```

DECOMPOSE_PROMPT = (
    "Break this multi-hop question into 2-3 ordered, self-contained sub-questions, "
    "one per line, no numbering. If it is already simple, return it unchanged.\nQuestion:
    {q}"
)

def detect_false_premise(query: str, llm: LocalLLM) -> bool:
    out = llm.chat("You detect false presuppositions.",
                   FALSE_PREMISE_PROMPT.format(q=query), max_tokens=4)
    return out.strip().lower().startswith("y")

#### OUTPUT ####
route('Were Scott Derrickson and Ed Wood of the same nationality?...') -> single_hop
decompose ->
    • What is the nationality of Scott Derrickson?
    • What is the nationality of Ed Wood?

```

The same router on two other kinds of question shows the other branches.

```

#### OUTPUT ####
route('What is the best programming language?') -> no_retrieval
route('Who directed Ed Wood, and what is that director also known for?') -> multi_hop

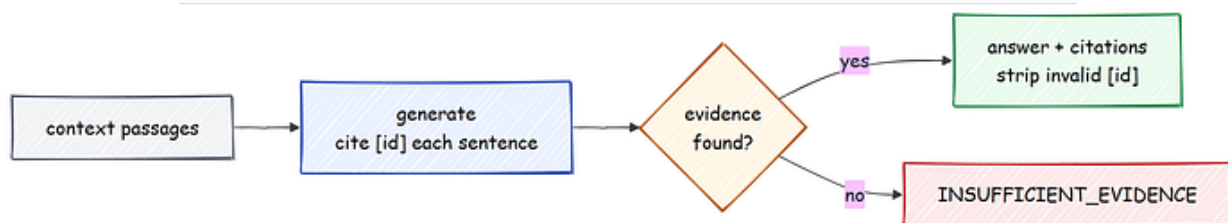
```

An opinion gets `no_retrieval`, which is itself an abstention path, because the system declines rather than search for an answer no document holds. A real two-fact question gets `multi_hop`, which is what later sends the agent into its corrective loop.

The router calls our running example single-hop because the reranked passages already answer it directly, and the decomposer still shows how it would break the comparison into two clean lookups if the first pass came back thin. Routing is cheap, a single short classification call, and it earns its place by keeping the expensive retrieval and verification work off the questions that do not need it, which also matters at scale because every retrieval I skip is latency I do not spend.

Cited Generation

This is the first hallucination firewall. The system prompt forbids outside knowledge, requires an inline citation for every sentence, and gives the model an explicit token to emit when the context does not contain the answer. Telling the model to cite is not enough on its own, so we also validate the citations and drop any the model invented.



```

ABSTAIN_TOKEN = "INSUFFICIENT_EVIDENCE"
GENERATION_SYSTEM_PROMPT = (
    "You answer strictly from the numbered context passages. Rules:\n"
    "1. Use ONLY facts in the passages, never outside knowledge.\n"
    f"2. If the passages do not contain the answer, reply with exactly: {ABSTAIN_TOKEN}\n"
    "3. Every sentence MUST end with a citation to the passage id(s) it uses, like\n"
    "[abc123def456].\n"
    "4. Be concise and factual."
)

```

After generation we parse the citation markers and keep only the ones that match a real chunk id, so a fabricated citation can never survive to the user.

```

def parse_citations(text: str, valid_ids: set[str]) -> tuple[list[str], str]:
    found = _CITE_RE.findall(text)
    valid = [c for c in dict.fromkeys(found) if c in valid_ids]
    invalid = [c for c in dict.fromkeys(found) if c not in valid_ids]
    cleaned = text
    for bad in invalid:
        cleaned = cleaned.replace(f"[{bad}]", "")
    return valid, cleaned

```

Run it on a sentence that cites one real passage and one the model invented, and the fake citation simply disappears.

```

#### OUTPUT ####
>>> text = "Paris is the capital of France [a1b2c3d4e5f6]. The Louvre opened in 1793 [deadbeef0000]."
>>> parse_citations(text, valid_ids={"a1b2c3d4e5f6"})
(['a1b2c3d4e5f6'], 'Paris is the capital of France [a1b2c3d4e5f6]. The Louvre opened in 1793 .')

```

The valid id stays and the invented [deadbeef0000] is stripped, so only a real citation reaches the next stage. This matters because the most dangerous hallucination is a confident sentence wearing a citation it did not earn, and here that citation is gone before anyone sees it. The generator formats the retrieved passages with their ids, calls the model once, and either returns the abstain signal or a parsed, citation-checked answer.

```

class CitedGenerator:
    def generate(self, question, chunks) -> CitedAnswer:
        user = f"Context passages:\n{format_context(chunks)}\n\nQuestion: {question}\n\nAnswer:"
        raw = self.llm.chat(GENERATION_SYSTEM_PROMPT, user, max_tokens=400).strip()
        if ABSTAIN_TOKEN in raw:
            # the model chose to abstain
            return CitedAnswer(text="", cited_ids=[], abstained=True, raw=raw)
        cited, cleaned = parse_citations(raw, {c.id for c in chunks})
        return CitedAnswer(text=cleaned.strip(), cited_ids=cited, abstained=False, raw=raw)
)

#### OUTPUT ####
Q: Were Scott Derrickson and Ed Wood of the same nationality?
abstained=False citations=['a9ec406223bd', '2d2201c92ac5']
A: Yes, Scott Derrickson and Ed Wood were of the same nationality; both were American.
[a9ec406223bd] [2d2201c92ac5]

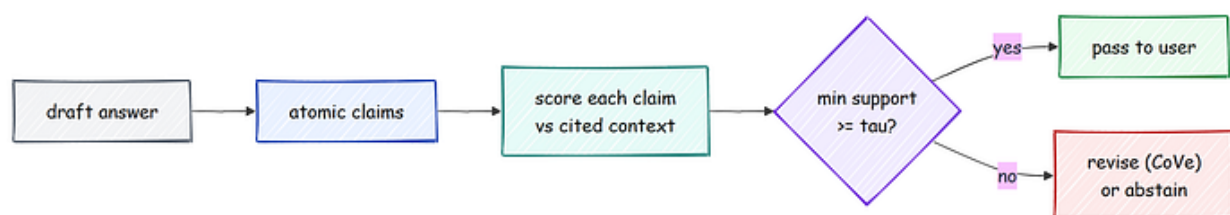
```

The answer cites the two passages we retrieved, and both ids are real, so nothing gets stripped. At this point we have a fluent, cited answer, but a citation only proves the model pointed at a passage, not that the passage actually supports what it said.

A model can cite a real passage and still misread it, so a citation is necessary but not sufficient. That gap is what the next firewall closes.

The Verification Gate

This is the decisive firewall. We split the drafted answer into atomic claims, then check each claim against its cited context with the faithfulness judge we loaded earlier. A claim that scores below the threshold is unsupported, and if any claim fails, the whole answer is downgraded to an abstention.



The claim extractor splits the answer into atomic, independently checkable statements, dropping the citation markers first so the claims are clean text.

```
class ClaimExtractor:
    def extract(self, answer: str) -> list[str]:
        clean = _CITE_RE.sub("", answer).strip() # remove [id] markers first
        out = self.llm.chat("You extract atomic factual claims.",
                             CLAIM_DECOMP_PROMPT.format(a=clean), max_tokens=300)
        claims = [re.sub(r"^\s*\d+[\.])\s*", "", ln).strip(" -\t")
                    for ln in out.splitlines() if ln.strip()]
        return [c for c in claims if len(c) > 3]
```

The gate extracts the claims, scores each against the cited passages, and passes only if every claim clears the threshold.

```
class VerificationGate:
    def check(self, cited: CitedAnswer, chunks: list[RetrievedChunk]) -> GateResult:
        claims = self.extractor.extract(cited.text) # split into atomic claims
        used = [c for c in chunks if c.id in set(cited.cited_ids)] or chunks
        context = "\n\n".join(c.text for c in used)
        verdicts = []
        for cl in claims:
            s = self.verifier.support(cl, context)
            verdicts.append(ClaimVerdict(cl, s["score"], s["score"] >= self.tau,
                                         s["nli"], s["minicheck"]))
        min_support = min((v.score for v in verdicts), default=0.0)
        passed = len(verdicts) > 0 and all(v.supported for v in verdicts)
        return GateResult(passed, verdicts, min_support, len(verdicts))

#### OUTPUT ####
claims=3 passed=True min_support=1.00
[OK 1.00] Scott Derrickson is American.
[OK 1.00] Ed Wood is American.
[OK 1.00] Scott Derrickson and Ed Wood share the same nationality.
```

The one-sentence answer breaks into three checkable claims, and each one scores a full 1.00 against the cited passages, so the gate passes with a minimum support of 1.00. Checking at the claim level instead of the whole answer is what makes this strict.

A long answer can be eighty percent grounded and still smuggle in one invented fact, and an answer-level score would wave it through, while a claim-level gate isolates that one sentence and fails on it. The key design choice is that the gate reports the **weakest** claim, not the average, because an answer is only as trustworthy as its least supported sentence.

That weakest-claim rule is best seen when it fires. Here is the same gate on a draft for one of the false-premise questions, where the model tried to oblige.

```
#### OUTPUT ####
claims=2 passed=False min_support=0.20
[OK 0.95] Marie Curie was a physicist.
[XX 0.20] Marie Curie traveled to the Moon.
```

The first claim is well supported, but the second scores 0.20, far below the 0.3 threshold, because no passage says any such thing. One failing claim flips `passed` to `False`, the whole answer is thrown out, and the question becomes an abstention instead of a confident false statement. This is the exact moment a hallucination is caught and turned into a safe refusal.

For a borderline answer we do not just throw it away. A chain-of-verification pass gives it one chance to repair itself, rewriting any sentence the context does not support and keeping the citations, and then the gate runs again on the revised text.

```
COVE_PROMPT = (
    "Revise the answer so EVERY sentence is directly supported by the context. "
    "Remove or soften any claim not supported. Keep citations [id].\n\n"
    "Context:\n{ctx}\n\nAnswer:\n{ans}\n\nRevised answer:"
)

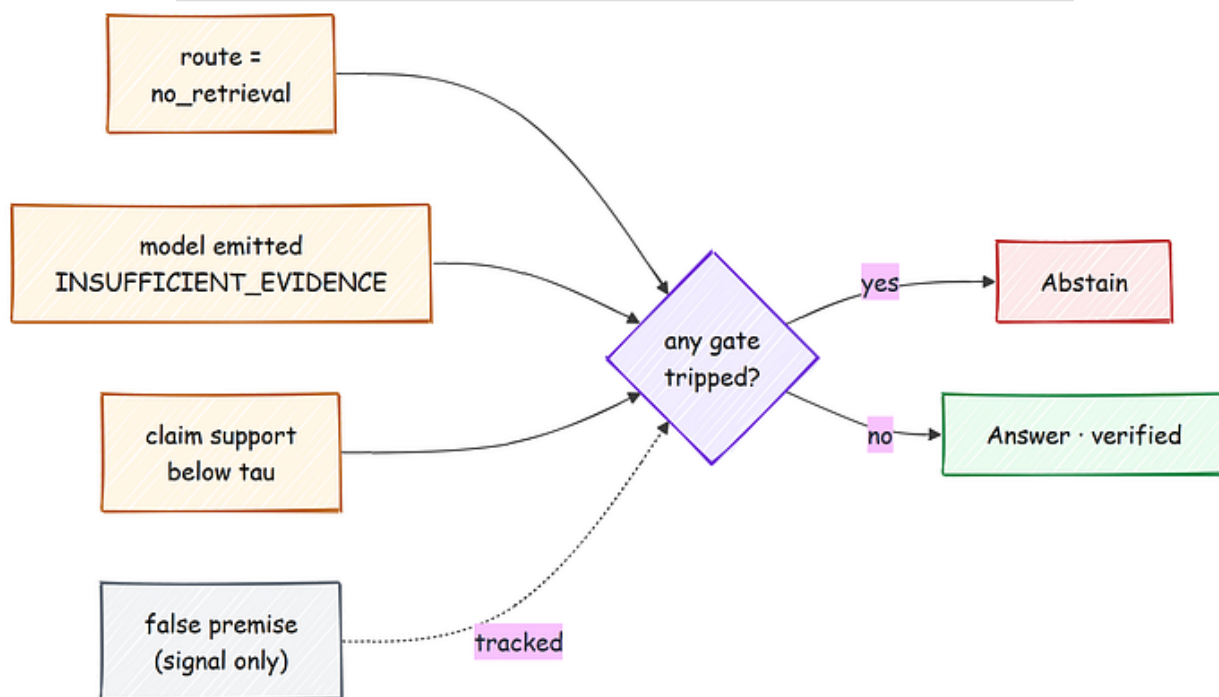
def cove_revise(answer: str, chunks, llm: LocalLLM) -> str:
    ctx = format_context(chunks)
    return llm.chat("You make answers strictly faithful to context.",
                    COVE_PROMPT.format(ctx=ctx, ans=answer), max_tokens=400).strip()
```

Knowing When to Abstain

Abstention is a correct answer, not a failure, so we make it a first-class outcome. This is the move that makes near-zero hallucination possible at all.

I cannot stop the model from being wrong on a question with no answer in the corpus, but I can make the system refuse that question, which turns an unbounded failure, a confident lie, into a bounded one, a visible abstention I can measure and tune. The policy folds the signals into one decision.

If the router said no retrieval, or the model emitted the abstain token, or the verification gate failed, we abstain, and otherwise we answer with the verified text.



Every outcome is one strict, auditable record, so evaluation can parse answered against abstained without any guesswork.

```

@dataclass
class FinalAnswer:
    status: str          # "answered" or "abstained"
    answer: str
    citations: list[str]
    min_support: float
    reason: str          # which gate fired, or "verified"

class AbstentionPolicy:
    def decide(self, route, false_premise, cited, gate) -> FinalAnswer:
        if route == "no_retrieval":
            return self._abstain("routed_no_retrieval", gate)
        if cited.abstained:
            return self._abstain("model_abstained", gate)
        if gate is None or not gate.passed or gate.min_support < self.tau:
            return self._abstain("unsupported_claims", gate)
        return FinalAnswer("answered", cited.text, cited.cited_ids,
                           gate.min_support, "verified", {})

#### OUTPUT ####
AbstentionPolicy ready; reasons = {routed_no_retrieval, false_premise, model_abstained,
unsupported_claims, verified}
  
```

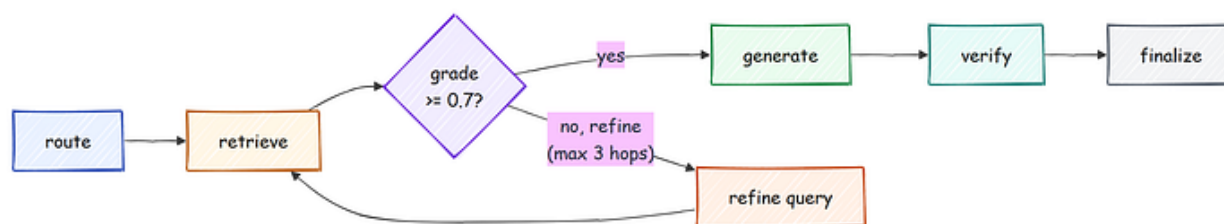
There is one subtlety worth calling out. The false-premise flag is recorded as a signal, but it is not a hard gate, because a small yes-or-no detector is too noisy to trust on its own.

We let the evidence path of grading plus claim verification make the real decision, which catches false-premise questions anyway when no passage supports them. When the system does abstain, it returns a plain message, "I do not have enough supporting evidence in the available sources to answer this confidently," instead of a guess.

The Agent

We have now built every component, so the last step is to wire them into a graph that corrects itself, because the single biggest cause of hallucination is generating from bad context. The loop is built with LangGraph, which I choose because the control flow is genuinely a graph and not a straight line, route can skip retrieval, grade can loop back through refine, and verify can downgrade an answer to an abstention, so I would rather declare those edges than bury them in nested conditionals.

We route, retrieve, then grade the evidence. If the evidence is strong we generate, if it is weak we refine the query and retrieve again up to a hop cap, and if it is hopeless we abstain without ever generating.



The agent passes one state object between nodes, a typed dictionary that accumulates the route, the evidence, the grade, the draft, the gate result, and a running latency tally.

```

class AgentState(TypedDict, total=False):
    question: str
    route: str
    query: str
    evidence: list
    grade: float
    draft: Any
    gate: Any
    final: Any
    hops: int
    latencies: dict
  
```

Each node does one job. The grader scores how well the current passages answer the question, and the refine node is the corrective step, it bumps the hop counter, decomposes the question, and widens the query before we retrieve again.

```
def grade_evidence(query: str, chunks, llm: LocalLLM) -> float:
    ctx = "\n".join(f"- {c.text[:200]}" for c in chunks[:8])
    out = llm.chat("You grade retrieval sufficiency.",
                   GRADE_PROMPT.format(q=query, ctx=ctx), max_tokens=8)
    m = re.search(r"[01](?:\.\d+)?", out)
    return float(m.group()) if m else 0.5

def n_refine(state: AgentState) -> AgentState:
    state["hops"] = state.get("hops", 0) + 1
    subs = decomposer.decompose(state["question"])
    state["query"] = " ".join(subs)          # broaden the query with the sub-questions
    return state
```

A small routing function turns the grade into the next move, and the graph wires the nodes together with the refine step looping back to retrieve.

```
def _after_grade(state: AgentState) -> str:
    g = state.get("grade", 0.0)
    if g >= CRAG_OK:                      # 0.7+, the evidence is strong, answer it
        return "generate"
    if g < CRAG_BAD or state.get("hops", 0) >= MAX_HOPS:
        return "generate" if g >= CRAG_BAD else "finalize" # too weak, abstain
    return "refine"                        # borderline, refine the query and retry

def build_agent_graph():
    g = StateGraph(AgentState)
    for name, fn in [("route", n_route), ("retrieve", n_retrieve), ("grade", n_grade),
                    ("refine", n_refine), ("generate", n_generate),
                    ("verify", n_verify), ("finalize", n_finalize)]:
        g.add_node(name, fn)
    g.set_entry_point("route")
    g.add_conditional_edges("grade", _after_grade,
                           {"generate": "generate", "refine": "refine", "finalize": "finalize"})
    g.add_edge("refine", "retrieve")        # the corrective loop
    g.add_edge("generate", "verify")
    g.add_edge("verify", "finalize")
    return g.compile()
```

Running the full agent over our running example shows every stage and its timing.

```
#### OUTPUT ####
Q: Were Scott Derrickson and Ed Wood of the same nationality?
route=single_hop hops=0 grade=1.00 status=answered reason=verified
A: Yes, Scott Derrickson and Ed Wood were of the same nationality; both were American.
latencies(s): {'route': 0.16, 'retrieve': 2.4, 'grade': 0.13, 'generate': 0.94, 'verify': 0.97, 'total': 4.6}
```

The grade comes back at 1.00, so the agent goes straight to generation, and the final status is `answered` with reason `verified`, which means it passed every gate we built. The hop counter stays at zero here, but on a thin retrieval it would climb to three before giving up. The bounded loop is what keeps latency in budget while still allowing a second and third try.

The contrast is the whole design in two lines. Send the agent a question with no answer in the corpus, and the same graph reaches the opposite, correct conclusion.

```
#### OUTPUT ####
Q: Which programming language did Isaac Newton invent in 1700?
route=single_hop hops=0 grade=0.15 status=abstained reason=unsupported_claims
A: I do not have enough supporting evidence in the available sources to answer this
confidently.
latencies(s): {'route': 0.17, 'retrieve': 2.9, 'grade': 0.14, 'total': 3.3}
```

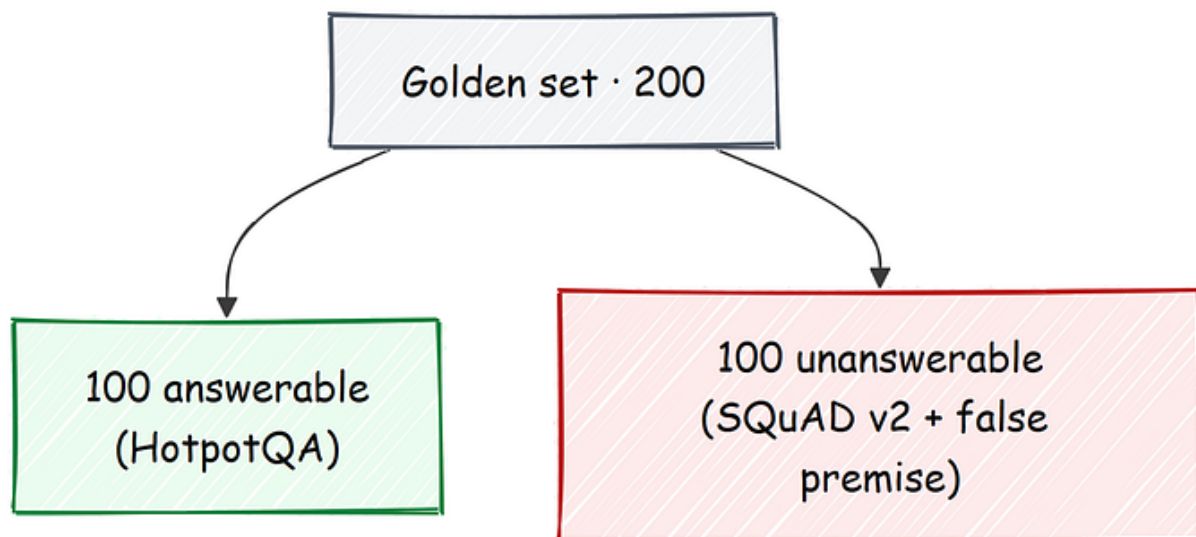
Retrieval finds nothing about Newton inventing a language, so the grade comes back at 0.15, below the `crag_bad` floor of 0.4, and the agent finalizes straight to an abstention without ever generating. That early exit is also why the abstain path is faster, 3.3 seconds here against 4.6 for the answered case, because the system spends nothing on generation or verification once it knows the evidence is not there. This is what the 98 out of 100 abstentions on the unanswerable set look like, one question at a time.

Does It Work?

The golden set

To measure any of this we need a test set with two strata. The answerable stratum comes from HotpotQA, and the unanswerable stratum comes from SQuAD v2 impossible questions plus a handful of hand-built false-premise questions.

The unanswerable half is the important one, because it is where a normal RAG system quietly bluffs. Everything we built, the citation rule, the claim gate, the abstention policy, exists to keep that half quiet, so this is the stratum that actually scores the near-zero hallucination claim, while the answerable half scores whether retrieval did its job.



```
def build_false_premise_set() -> list[EvalItem]:
    qs = [
        "In what year did Albert Einstein win his second Nobel Prize in Physics?",
        "What was the name of the spaceship Marie Curie flew to the Moon?",
        "How many gold medals did William Shakespeare win at the Olympics?",
        "Which programming language did Isaac Newton invent in 1700?",
    ]
    return [EvalItem(f"fp_{i}", q, "", [], False, "false_premise") for i, q in enumerate(qs)]

#### OUTPUT ####
[golden] 200 items (answerable=100, unanswerable=100)
```

We end up with a balanced **200 question** set, half answerable and half not. The false-premise questions are deliberately absurd, like asking which language Newton invented in 1700, because a system that answers those is a system that will invent facts for any confident-sounding question.

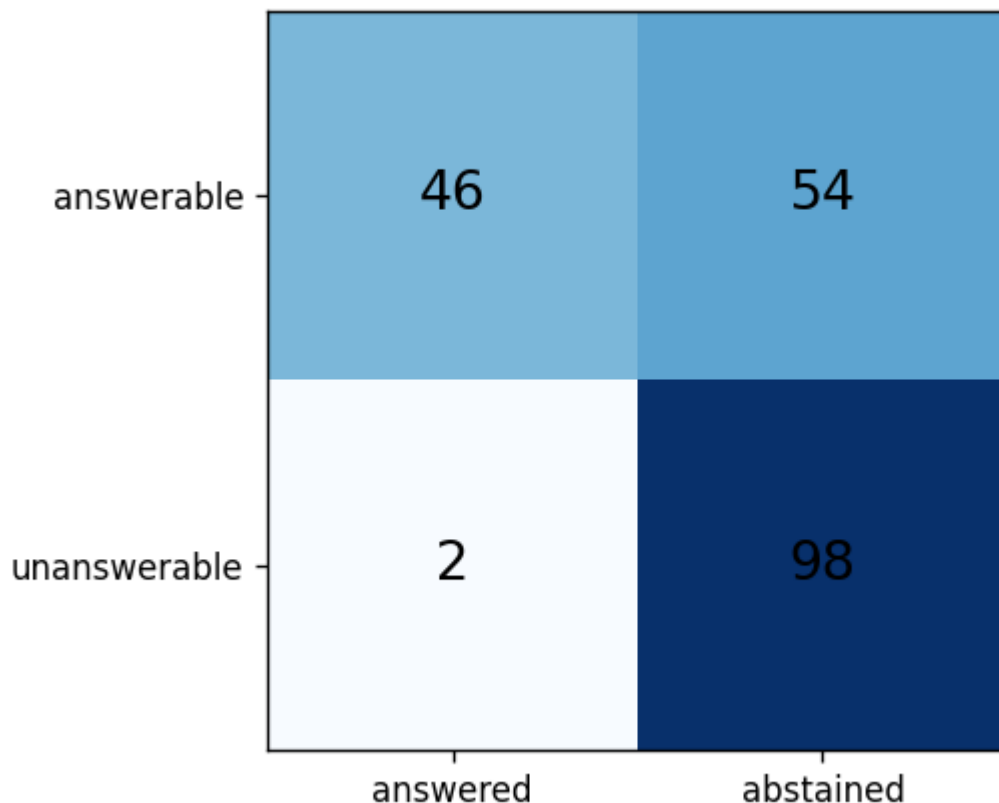
Balancing the two halves matters, because a set that is mostly answerable would let a system score well while still bluffing on the hard cases. Half of this set exists purely to measure restraint.

Hallucinations live in one cell

Now we run the agent over all 200 questions and score the result as a two-by-two table. The rows are answerable or unanswerable, the columns are answered or abstained, and the one dangerous cell is unanswerable and answered, because that is a hallucination by definition.

```
def confusion_2x2(results, items) -> np.ndarray:
    cm = np.zeros((2, 2), dtype=int) # rows: answerable/unanswerable, cols: answered/abstained
    for r, it in zip(results, items):
        i = 0 if it.answerable else 1
        j = 0 if r.final.status == "answered" else 1
        cm[i, j] += 1
    return cm
```

2x2: hallucinations live in (unanswerable, answered)



```
#### OUTPUT ####
confusion (rows ans/unans, cols answered/abstained):
[[46 54]
 [ 2 98]]
hallucinations (unanswerable answered): 2 / 100 unanswerable
```

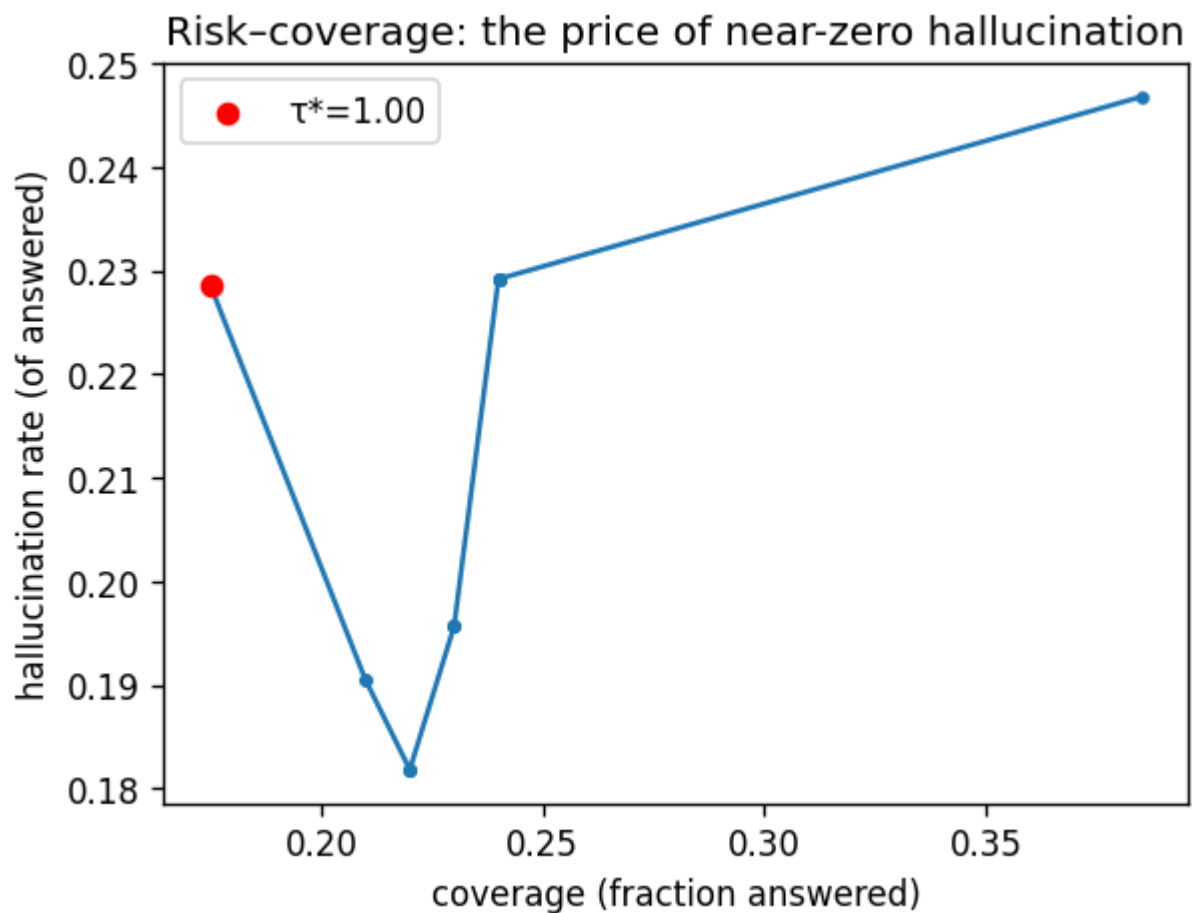
Read the bottom row, because it is the whole point. Of the 100 unanswerable questions, the system abstained on **98** and only answered **2**, which is a **2 percent** hallucination rate on the questions designed to trap it.

A plain RAG system with no verification gate would light up that cell instead, because nothing would stop it from answering a question the corpus cannot support. The top row of the matrix is what this safety costs us, and we look at it next.

The price of safety

The two-by-two used one fixed threshold, but the threshold is a dial. Turn it up and the system abstains more, which lowers hallucination but also lowers coverage. To choose it deliberately we sweep the threshold and draw a risk-coverage curve, then pick the point that keeps hallucination under a budget while answering as much as possible.

```
def pick_tau(df, max_halluc: float = 0.05) -> float:
    # among thresholds that keep hallucination under the budget, take the most coverage
    ok = df[df["hallucination_rate"] <= max_halluc]
    return float(ok.sort_values("coverage", ascending=False).iloc[0]["tau"]) if len(ok) else 1.0
```



```
#### OUTPUT ####
chosen  $\tau^*$  (halluc $\leq$ 5%): 1.0
metrics: {
  "faithfulness": 0.908,
  "answer_relevancy": 0.817,
  "context_recall@k": 0.97,
  "answerable_accuracy": 0.58
}
```

On answered questions we get **0.908 faithfulness** and **0.97 context recall**, which says the evidence is there and the answers stay grounded in it. The price is the top row of the matrix.

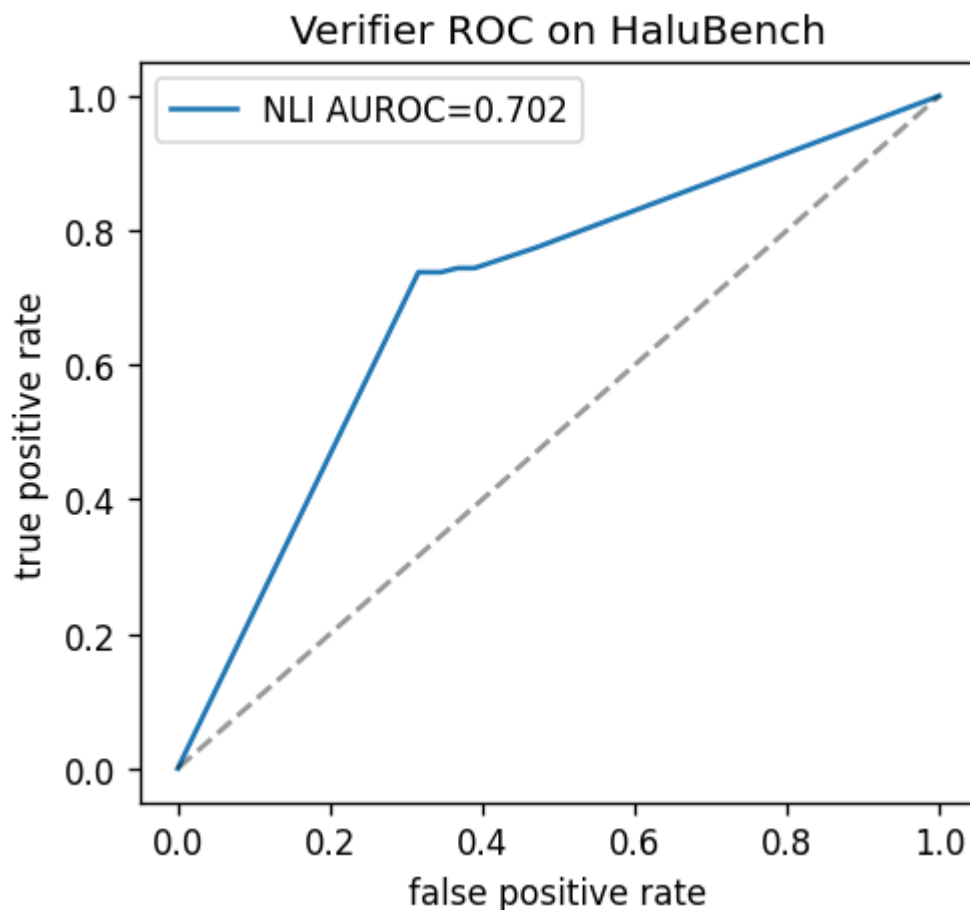
We answer **46** of the 100 answerable questions and abstain on the rest, a coverage of 0.46. That is the deliberate trade.

We would rather stay silent on a question we could have answered than risk a confident wrong answer. Where exactly you sit on this curve is a product decision and not a model one, and it can be set per corpus depending on how expensive a wrong answer is in that domain.

Is the judge any good?

There is a hole to close. The whole gate leans on the verifier, so an unverified verifier just moves the hallucination from the answer into the scorecard. We test the verifier on its own against HaluBench, a set of human-labeled faithful and hallucinated answers, and report the area under the ROC curve.

```
def eval_verifier(verifier, n: int = 300) -> dict:
    hb = load_halubench().shuffle(seed=SEED).select(range(n))
    scores, labels = [], []
    for ex in hb:
        scores.append(verifier.nli_score(ex["answer"], ex["passage"])) # the judge's
        support score
        labels.append(1 if str(ex["label"]).upper().startswith("PASS") else 0)
    from sklearn.metrics import roc_auc_score
    return {"auROC": round(float(roc_auc_score(labels, scores)), 3), "n": len(labels)}
```



```
#### OUTPUT ####
```

```
[verifier] AUROC=0.702 over n=300 HaluBench items
```

The verifier scores an **AUROC of 0.702** over 300 items, which is clearly better than chance but a long way from perfect. I want to be plain about that, because it is the real ceiling on the whole gate.

A stronger verifier is the single change that would push the numbers above further, and the architecture is built so we can drop one in without touching the rest. The gate does not need a perfect verifier to help, it needs one that ranks supported claims above unsupported ones often enough to move the operating point, and 0.702 clears that bar while leaving plenty of room to grow.

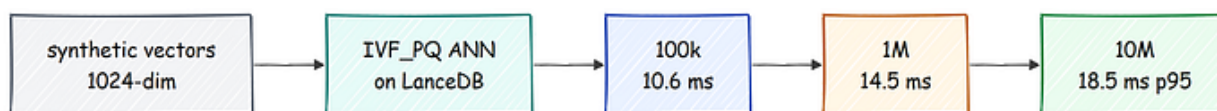
Scaling to 10M+ Vectors

A real 10M-vector index

The quality pipeline is proven on a curated slice. Now we have to prove the scale claim literally, because the title says 10M+ documents and a benchmark is the only thing that settles it.

So we build a LanceDB index at 100k, 1M, and 10M vectors, with a real approximate nearest neighbor index, and we measure build time, on-disk size, and query latency at each step. I have to use an approximate IVF_PQ index rather than exact search, because an exact scan compares the query against every vector and is linear in n , which is exactly the cost that explodes at 10M, while an approximate index visits only a few partitions and quantizes each vector down to a few bytes, trading a little recall for latency that barely moves as the corpus grows.

To keep this a clean vector-search benchmark, the vectors here are synthetic, 1024-dimensional unit vectors, and we ingest them through Arrow so the path holds tens of millions of rows. The host has 180 GB of RAM and a 750 GB NVMe disk, so a ten million vector index fits comfortably on one machine, which is the entire point of an on-disk store.



```

class ScaleBench:
    def run(self, sizes: list[int]) -> "pd.DataFrame":
        rows = []
        for n in sizes:
            vecs = make_synthetic_vectors(n, self.dim) # 1024-dim unit vectors
            db = lancedb.connect(str(SCRATCH_DIR / f"scale_{n}"))
            t0 = time.time()
            tbl = db.create_table("v", data=self._arrow(vecs), mode="overwrite")
            if n >= 100_000: # build a real ANN index
                tbl.create_index(metric="cosine",
                                num_partitions=int(min(4096, max(256, n ** 0.5))),
                                num_sub_vectors=64)
            build_s = time.time() - t0
            # then time 50 queries for p50/p95 and check recall@10 against brute force
            rows.append(self._measure(tbl, vecs, build_s))
        return pd.DataFrame(rows)

#### OUTPUT ####
[scale] building n=100,000 with IVF_PQ ANN index ...
-> {'n': 100000, 'build_s': 41.82, 'disk_gb': 0.39, 'p50_ms': 8.5, 'p95_ms': 10.59, 'recall@10': 0.135}
[scale] building n=1,000,000 with IVF_PQ ANN index ...
-> {'n': 1000000, 'build_s': 81.22, 'disk_gb': 3.884, 'p50_ms': 11.34, 'p95_ms': 14.46, 'recall@10': 0.105}
[scale] building n=10,000,000 with IVF_PQ ANN index ...
-> {'n': 10000000, 'build_s': 347.04, 'disk_gb': 38.825, 'p50_ms': 16.91, 'p95_ms': 18.48, 'recall@10': 0.105}
  
```

The headline is in the last line. A **10M-vector** index answers at **18.48 ms p95**, while the index from a hundred times fewer vectors answers at 10.59 ms.

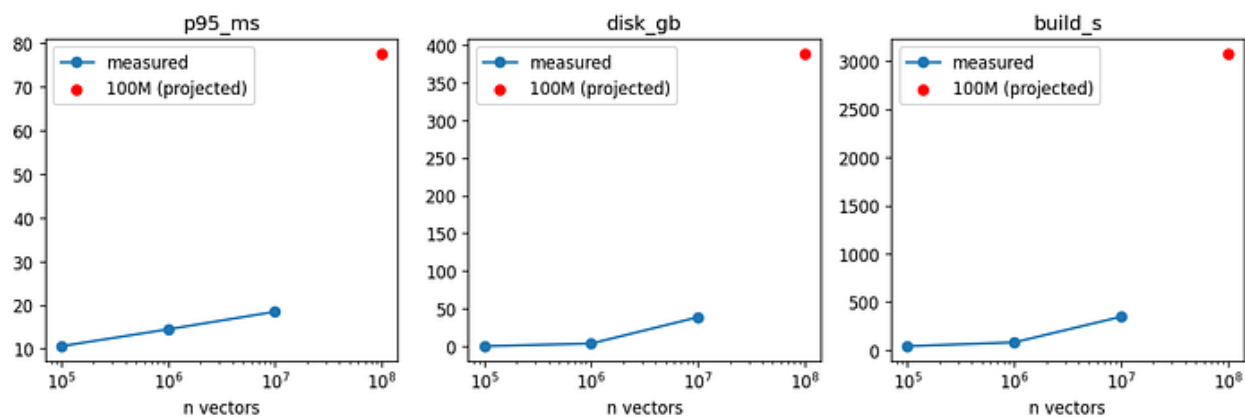
A hundredfold growth in the data cost us less than a doubling in latency. The disk grows linearly, from 0.39 GB to 38.8 GB, which is exactly what we want, because disk is cheap and an in-memory index at this size would not be.

Build time grows the same gentle way, from 42 seconds at a hundred thousand vectors to under six minutes at ten million, and every byte of it stays on the NVMe disk of one machine.

18 ms at ten million, and a 100M projection

The reason latency barely moved is the nature of an approximate index. An IVF_PQ index searches a few partitions instead of the whole space, so query cost grows with the number of partitions, not with the number of vectors, while disk grows linearly because every vector still has to be stored. We fit that trend and project it to 100M.

```
def fit_and_extrapolate(df, target: int = 100_000_000) -> dict:
    n = df["n"].values.astype(float)
    out = {"target": target}
    for col in ["build_s", "disk_gb", "p95_ms"]:
        a, b = np.polyfit(n, df[col].values, 1) # linear fit in n
        out[col] = round(float(a * target + b), 2)
    return out
```



```
#### OUTPUT ####
projection to 100M: {
    "build_s": 3075.1,
    "disk_gb": 388.23,
    "p95_ms": 77.58
}
```

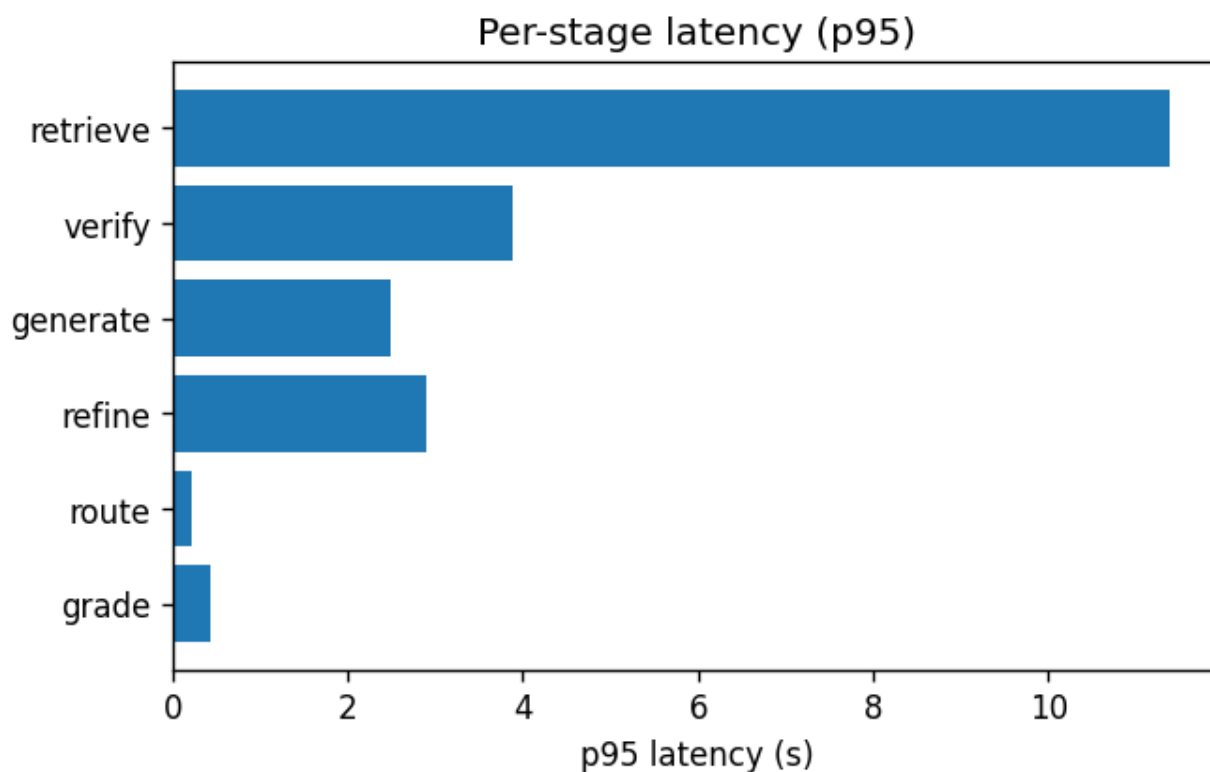
At **100M vectors** the projection lands at **77.58 ms p95** with a 388 GB index, which still fits on the NVMe disk of a single box. One caveat stated plainly.

Recall at 10 sits near 0.1 here only because the vectors are random, which gives an approximate index almost nothing real to find, so this run measures latency and throughput, not retrieval quality. On a real corpus the same index keeps recall high, and the latency numbers are what hold as you scale.

Where the time goes

Scale is the easy part. The expensive part is the per-query agent, so we attribute latency by stage to see where the budget actually goes.

```
def aggregate_latencies(results) -> "pd.DataFrame":
    stages = {}
    for r in results:
        for k, v in r.latencies.items():
            stages.setdefault(k, []).append(v)
    rows = [{"stage": k, "p50_s": round(np.percentile(v, 50), 3),
            "p95_s": round(np.percentile(v, 95), 3),
            "mean_s": round(np.mean(v), 3)} for k, v in stages.items()]
    return pd.DataFrame(rows).sort_values("mean_s", ascending=False)
```




```
#### OUTPUT ####
```

stage	p50_s	p95_s	mean_s
total	4.001	17.668	5.823
retrieve	3.074	11.393	4.166
verify	1.534	3.878	1.758
generate	1.451	2.484	1.619
refine	1.471	2.888	1.575
route	0.168	0.206	0.170
grade	0.127	0.431	0.161

A typical question finishes in **4 seconds** at the median, and the slow tail reaches 17.7 seconds at p95. **Retrieve dominates**, because it runs the embedder, both searches, and the cross-encoder reranker over 150 candidates, and on hard questions it runs more than once through the corrective loop.

The vector search itself is the cheap part, which is the same lesson the scale lab taught. The index is not the bottleneck, the language model calls around it are.

That is worth knowing before optimizing, because it means the wins live in cutting model calls, batching the reranker, or caching grades, not in a faster vector store.

Scope and What Comes Next

I want to close by being plain about what this is and what it is not. The hallucination rate is **2 percent on the unanswerable set, not zero**, because literal zero is not achievable from a generative model.

Coverage on answerable questions is **0.46**, which is the deliberate price we pay for that safety, and the risk-coverage curve is the dial for trading one against the other. The 10M run is a vector-search benchmark on synthetic vectors, so it proves the index scales in latency and disk, while a real corpus is what keeps recall high at the same speed.

The verifier sits at **AUROC 0.702**, which is good but not great, and it is the most valuable thing to improve next.

From here, a few directions are worth the effort.

1. **A stronger verifier:** the gate is only as good as the judge, so a better faithfulness model lifts every downstream number at once.
2. **Real embeddings at scale:** rerun the scale lab over real document vectors to confirm recall holds while the 18 ms latency stays put.
3. **Sharding and quantization:** past a single box, the index splits across shards, and the correctness logic above does not change at all.

4. **Calibrated coverage:** tune the thresholds per domain so high-stakes corpora abstain more and casual ones answer more.

None of these next steps change the spine of the design. The index can grow, the verifier can improve, and the thresholds can move, but the contract stays the same. Every sentence that reaches a user is one the system could point to in the retrieved text, and everything else becomes an abstention.

The whole thing is one idea carried all the way through. We do not try to make the model never wrong, we build a system that only ever says what it can prove, and abstains otherwise. The index scales to ten million vectors at 18 ms, the answers stay grounded at 0.908 faithfulness, and the questions it cannot support come back as a plain "I do not have enough evidence" instead of a confident guess.

The full notebook, with every code cell and the real run outputs, is on GitHub:

GitHub - FareedKhan-dev/rag-zero-hallucinations: Handling 10M+ docs using RAG with zero... (<https://github.com/FareedKhan-dev/rag-zero-hallucinations>)

Handling 10M+ docs using RAG with zero hallucinations - GitHub - FareedKhan-dev/rag-zero-hallucinations: Handling 10M+...

Wanna chat about RAG or anything else? Reach me on [my LinkedIn](https://www.linkedin.com/in/fareed-khan-dev/) (<https://www.linkedin.com/in/fareed-khan-dev/>).